FUNDAMENTALS OF PROGRAMMING I · 2026/27 · PROJECT 1

Document Classification

Deciding the topic of 40 news articles from their keywords, in Python

---

[Name Surname] · NIU [XXXXXXX]
[Name Surname] · NIU [XXXXXXX]

THE PROBLEM

Which topic is each article about?

For every article we count how many times the keywords of each category appear. If one category has **more than 50 %** of all the keyword occurrences, that is the article's topic; otherwise it is **UNKNOWN**.

Ex.	Function	Receives	Returns
2	Read_Categories_Keywords	keywords file name	l_categories, l_keywords
3	Document_Reader	one news file	l_words
4	Count_Frequencies	words + categories	d_frequencies
5	Classify_Doc	d_frequencies	d_votes, doc_category
6	main	file + folder names	the two output .txt files

DATA STRUCTURES

The types the statement asks for

Name	Type	Example
l_categories	list of str	['football', 'space', 'health', ...]
l_keywords	list of sets	[{'fifa', 'club', ...}, {'nasa', 'orbit', ...}, ...]
l_words	list of str	['once', 'rocket23', 'reached', 'the', ...]
d_frequencies	dict of dicts	{'space': {'nasa': 2, 'orbit': 0, ...}, ...}
d_votes	dict of tuples	{'space': (3, 75), 'sports': (1, 25), ...}
doc_category	str	'space' or 'UNKNOWN'

Parallel lists: position i of l_categories and position i of l_keywords describe the same category.

HELPER FUNCTION · EX. 1

Reading a file in two encodings

```
def Read_Text_File(filename):
    try:
        f = open(filename, "r", encoding="utf-8")
    except:
        raise FileNotFoundError
    try:
        text = f.read()
    except UnicodeDecodeError:
        f.close()
        f = open(filename, "r",
             encoding="windows-1252")
        text = f.read()
    f.close()
    return text
```

36 news files are UTF-8; **news_01, 10, 18 and 26** are Windows-1252.

Reading those as UTF-8 raises **UnicodeDecodeError**, so we reopen them as Windows-1252.

A file that can’t be opened raises **FileNotFoundError**, the statement’s model.

Written once, used by Ex. 2 and Ex. 3. Ex. 1 (DocClass_Welcome) only prints the message, copied exactly.

EXERCISE 2

Read_Categories_Keywords(filename)

```
def Read_Categories_Keywords(filename):
    l_categories = []
    l_keywords = []
    text = Read_Text_File(filename)
    for line in text.split("\n"):
        if ":" in line:
            parts = line.split(":")
            l_name = parts[0].split()
            if len(l_name) > 0:
                l_categories.append(l_name[0].lower())
                l_keywords.append(
                    set(parts[1].lower().split()))
    return l_categories, l_keywords
```

One line, step by step

`"space: planet orbit nasa  "`

split(":") → ['space', ' planet orbit nasa ']

parts[0].split() → ['space']

parts[1].split() → ['planet', 'orbit', 'nasa']

split() with no argument cuts at **1 or more spaces** and ignores the ones at the end.

EXERCISE 3

Document_Reader(filename)

```
def Document_Reader(filename):
    text = Read_Text_File(filename)
    clean = ""
    for c in text:
        if c.isalnum():
            clean = clean + c.lower()
        else:
            clean = clean + " "
    l_words = clean.split()
    return l_words
```

Letters and digits are **kept** (lowercase); **any other character** becomes a space.

```
NASA-team couldn't
→ "nasa team couldn t"
→ ['nasa', 'team', 'couldn', 't']
```

The statement's list of separators ends in "etc.", so we define what a word **is** (isalnum) instead of listing separators.

EXERCISE 4

Count_Frequencies(l_words, l_categories, l_keywords)

```
d_frequencies = {}
for i in range(len(l_categories)):
    d_frequencies[l_categories[i]] = {}
    for keyword in l_keywords[i]:
        d_frequencies[l_categories[i]][keyword] = 0
for word in l_words:
    for i in range(len(l_categories)):
        if word in l_keywords[i]:
            d_frequencies[l_categories[i]][word] += 1
return d_frequencies
```

1. Every keyword of every category starts at **0** (“0 if it does not appear”, and no KeyError).

2. One pass over the words: **in** on the set says if the word is a keyword.

Whole words only: **teams** is not **team**.

EXERCISE 5

Classify_Doc(d_frequencies)

```
# first loop (not shown): d_totals and total_all
d_votes = {}
doc_category = "UNKNOWN"
for category in d_totals:
    if total_all > 0:
        exact_percentage = d_totals[category] * 100 / total_all
    else:
        exact_percentage = 0
    d_votes[category] = (d_totals[category],
                         round(exact_percentage))
    if exact_percentage > 50:
        doc_category = category
return d_votes, doc_category
```

total_all > 0: no division by zero; with no keywords everything is 0 %.

The tuple stores the **rounded** %, the decision uses the **exact** one: 50.4 % wins, 50 % does not.

Only one category can pass 50 %, so it is always the highest.

EXERCISE 6 · STEPS 1–4

Main: ask until the input is valid

```
DocClass_Welcome()
l_categories = []
while len(l_categories) == 0:
    keywords_file = input("Enter the name of ...: ")
    try:
        l_categories, l_keywords = \
            Read_Categories_Keywords(keywords_file)
        if len(l_categories) == 0:
            print("Error: the file contains no data.")
    except FileNotFoundError:
        print("Error: the file does not exist.")
folder_name = input("Enter the name of the folder ...: ")
while not Folder_Has_Txt(folder_name):
    print("Error: ...")
    folder_name = input("Enter the name of the folder ...: ")
```

“Ask again until valid” = a **validation while**.

File: missing (**FileNotFoundError**) or empty (no categories).

Folder: **Folder_Has_Txt** searches for the first txt file and stops there.

EXERCISE 6 · STEP 5

Main: classify every file of the folder

```
for file_name in sorted(os.listdir(folder_name)):
    file_path = os.path.join(folder_name, file_name)
    if os.path.isfile(file_path):
        if Is_Txt(file_name):                    # file_name[-4:].lower() == ".txt"
            l_words = Document_Reader(file_path)      # Ex. 3, 4, 5 (inside a try)
            d_frequencies = Count_Frequencies(l_words, l_categories, l_keywords)
            d_votes, doc_category = Classify_Doc(d_frequencies)
            out.write(Line_Of_Document(file_name, d_votes, doc_category, d_frequencies) + "\n")
        else:
            out.write("Attention: the following non txt file has been detected: " + file_name + "\n")
```

```
news_01.txt: music(football:0%, space:0%, …, music:100%, aviation:0%): album:0, band:0, …, song:2, songs:0
```

EXERCISE 6 · STEP 6

Main: the statistics and the top 10

```
# l_names: keywords in alphabetical order
# l_counts: their totals (parallel list)
k = 0
while k < 10 and len(l_names) > 0:
    pos = 0
    for i in range(1, len(l_counts)):
        if l_counts[i] > l_counts[pos]:
            pos = i
    stats.write(l_names[pos] + ": " +
                str(l_counts[pos]) + "\n")
    del l_names[pos]
    del l_counts[pos]
    k += 1
```

Percentage of documents per category, in the file's order, with **"{0:.2f}".format(x)**.

Top 10: find the **maximum**, write it, delete it, 10 times.

Strict **>** keeps the first of a tie; the list is alphabetical, so ties come out alphabetically.

RESULTS

40 txt files: 38 classified, 2 UNKNOWN

docsNews2026_DCStatistics.txt

Category	Documents
football	15.00%
space	12.50%
health	15.00%
weather	15.00%
elections	15.00%
music	12.50%
aviation	10.00%
UNKNOWN	5.00%

Top keywords of the folder

Keyword	Occurrences
space	73
club	27
flight	23
nasa	20
election	16
launch	15
...	...
covid (10th)	11

PART 2

Other data structures we could use

For each structure of the project: an alternative, what it gains, and what it costs.

ALTERNATIVES

Each structure, and what could replace it

Now	Alternative	Advantage	Disadvantage
l_keywords	list of lists	keeps the file’s order	in compares one by one
2 parallel lists	dict category → set	can’t get out of step	no positions
d_frequencies	nested list freq[i][j]	only lists, fast by position	i, j must be remembered
d_frequencies	list of (cat, kw, n)	flat, easy to print	finding a count = full search
d_votes	2 parallel lists	no tuples needed	more lists to keep aligned
l_words	dict word → count	each word counted once	loses the word order

The statement’s structures win where it matters most: **in** on a set answers at once, and a dictionary keeps each count next to its name.

ALTERNATIVE, WRITTEN AND TESTED

d_frequencies as a nested list

```
def Count_Frequencies_List(l_words, l_kw_lists):
    l_freq = []
    for i in range(len(l_kw_lists)):
        l_freq.append([0] * len(l_kw_lists[i]))
    for word in l_words:
        for i in range(len(l_kw_lists)):
            for j in range(len(l_kw_lists[i])):
                if l_kw_lists[i][j] == word:
                    l_freq[i][j] += 1
    return l_freq
```

news_01, category music:

```
['singer', 'music', 'band', 'song', …]
[1, 0, 0, 2, 0, 0, 0, 0]
```

Same counts as d_frequencies (checked).

+ only lists: no dictionaries needed.

– a third loop to find each keyword, and the numbers mean nothing without the keyword list next to them.

TESTING

How we tested it, and what was hard

Tests

- Every example of the statement, Ex. 2 to 5: same output
- Wrong file and folder names: asks again
- A different keywords file with 3 categories
- Top 10 recounted with a separate method

Difficulties

- Two encodings: UnicodeDecodeError on 4 files
- Separators: defining a word with isalnum
- Exact output formats for the corrector
- Rounded vs exact percentage at 50 %

Thank you

Questions?

[Name Surname] · [Name Surname]